Assignment 1: Memory-Efficient Medical Data Streaming

Course: Programming for Artificial Intelligence

Dataset: PTB-XL ECG Database (Clinical Metadata)

1. Objective

The goal of this assignment is to build a robust data pipeline using **Generators** and **Iterators**. You will process a large-scale clinical dataset to filter and reconstruct patient records. This task simulates real-world AI workflows where datasets are too large to be loaded into memory at once.

2. Dataset Setup

- **Source:**
  - Complete dataset is available at: https://tinyurl.com/ptb-xl-complete
  - The CSV files are available at: https://tinyurl.com/ptb-xl-csv
- **File 1 (`ptbxl_database.csv`):** Contains clinical metadata for over 21,000 ECG records.
- **File 2 (`scp_statements.csv`):** Contains the mapping of SCP (Standard Communications Protocol) codes to human-readable diagnoses.

3. Requirements

Task 1: The SCP Mapping Dictionary

Before streaming the patient records, you must create a lookup table.

- Open `scp_statements.csv` using the `csv` module.
- Build a dictionary where the **Key** is the acronym (found in the first column) and the **Value** is the description (found in the "description" column).
- **Constraint:** Only include entries where the **"Statement Category"** is exactly `"Myocardial Infarction"`.

Task 2: The Filtered Patient Generator

Write a generator function `mi_patient_streamer(csv_path, mapping_dict)` that:

1. Uses `csv.DictReader` to open and read `ptbxl_database.csv`.
2. Iterates through the rows one by one.
3. **The Filter Logic:** The `scp_codes` column contains a string formatted like a dictionary (e.g., `"{'IMI': 35.0, 'SR': 0.0}"`). If any code from your MI mapping dictionary appears in this string, the record is considered a match.
4. **Yield:** For every match, yield a dictionary with the following keys:
   - `patient_id`, `age`, `sex`
   - `record_date` (from the `recording_date` column)
   - `diagnosis` (The full description retrieved from your mapping dictionary)
   - `filename` (from the `filename_hr` column)

Task 3: Execution & Timing

Use a simple loop to call your generator and print the first 5 matching records. Use the `time` module and appropriate decorator to measure how long it takes to find these records.

4. Implementation Details & Constraints

- **Allowed Libraries:** You **must** use the built-in `csv` module for file handling.
- **Forbidden Libraries:** Use of `Pandas`, `NumPy`, or `Polaris` is strictly prohibited.
- **Memory Efficiency:** The function **must** use the `yield` keyword. Creating a list inside the function and returning it at the end will result in a 50% grade deduction.
- **Parsing Hint:** Since the `scp_codes` column is a string, you can check for a code's existence using a simple membership test: `if f"'{code}'" in row['scp_codes']:`.

Sample Output Format

Your generator should yield dictionaries structured like this:

```
{
    'patient_id': 11275,
    'age': 48,
    'sex': 0,
    'record_date': '1984-12-01 14:49:52',
    'diagnosis': 'inferior myocardial infarction',
    'filename': 'records500/00000/00008_hr'
}
```

To help you maintain consistency and focus on the core logic of **Generators** and **Mapping**, here is a starter code template. It includes the necessary headers, function signatures, and a clear execution block.

```
import csv
import time

# --- TASK 1: THE SCP DECODER ---
def get_mi_mapping(mapping_csv_path):
    """
    Reads scp_statements.csv and returns a dictionary mapping
    codes to descriptions for 'Myocardial Infarction' only.
    """
    mi_map = {}
    # TODO: Implement CSV reading and dictionary building
    return mi_map

# --- TASK 2: THE PATIENT GENERATOR ---
def mi_patient_streamer(database_csv_path, mi_mapping):
    """
    A generator that yields patient records matching MI criteria.
    """
    # TODO: Use csv.DictReader to iterate through database_csv_path
    # TODO: Check if scp_codes contains any key from mi_mapping
    # TODO: Yield a dictionary with the required fields
    pass

# --- TASK 3: EXECUTION & TIMING ---
def main():
    db_path = 'ptbxl_database.csv'
    statements_path = 'scp_statements.csv'

    print("--- Initializing MI Mapping ---")
    mi_mapping = get_mi_mapping(statements_path)
    print(f"Loaded {len(mi_mapping)} MI-related diagnostic codes.\n")

    print("--- Starting Patient Stream ---")
    start_time = time.time()

    # Initialize the generator
    patient_gen = mi_patient_streamer(db_path, mi_mapping)

    # TODO: Print the first 5 matching records using the generator
    # Hint: Use next() or a loop with a counter

    end_time = time.time()
    print(f"\nTime taken to find records: {end_time - start_time:.4f} seconds")

if __name__ == "__main__":
    main()
```

Tips:

1. **File Paths:** Remember to place the CSV files in the same directory as the script or update the `db_path` and `statements_path` variables accordingly.
2. **String Membership:** Since you are using `csv.DictReader`, you should remember that `row['scp_codes']` is a string. You can use the `in` operator to check if a specific code exists within that string.
3. **Efficiency:** You are encouraged to use `break` inside the generator's inner loop once a match is found to avoid yielding the same patient multiple times if they have multiple MI-related codes.